

ЛАБОРАТОРНАЯ РАБОТА №10

«Управление эволюцией приложений с AI-ассистентом»

1. ЦЕЛЬ И ЗАДАЧИ РАБОТЫ

1.1. Цель

Научиться критически оценивать сгенерированный код, управлять его эволюцией при изменении требований и принимать архитектурные решения, обеспечивающие гибкость и надежность веб-приложений с базой данных.

1.2. Задачи

После выполнения лабораторной работы обучающийся будет:

Знать:

- принципы вайб-программирования в контексте веб-разработки;
- стратегии миграции API и схемы базы данных;
- анти-паттерны в AI-сгенерированном коде (нарушение слоистой архитектуры, избыточная логика в контроллерах, игнорирование транзакционности);

Уметь:

- анализировать разрыв между старой и новой спецификацией;
- формулировать промпты для рефакторинга существующего кода, а не для написания его с нуля;
- проектировать миграцию API и схемы БД с сохранением обратной совместимости;
- определять, что из сгенерированного кода можно оставить, а что переделать;
- документировать процесс использования AI-инструментов и принятые решения.

2. ТЕОРЕТИЧЕСКАЯ ЧАСТЬ

2.1. Вайб-программирование в веб-разработке

Вайб-программирование (Vibe Programming) — это подход, при котором разработчик формулирует **намерение (Intent)** на естественном языке, а AI-инструмент генерирует код. Для веб-приложений это особенно эффективно, так как паттерны (Controller-Service-Repository, REST API, CRUD) хорошо формализуемы и часто встречаются в обучающих данных AI.

Основной навык — умение переводить требования в **конкретные, измеримые промпты**, учитывающие слоистую архитектуру, работу с БД и безопасность.

Важно: При изменении требований разработчик **не начинает всё с нуля**, а анализирует текущий код, определяет, что можно сохранить, и формулирует промпты для целенаправленного рефакторинга.

2.2. Стратегии миграции API и БД

Тип изменения	Стратегия	Пример
Изменение API	URL-версионирование	<code>/api/v1/courses</code> → <code>/api/v2/courses</code>
Изменение параметров	Path → Query параметры	<code>/courses/{id}</code> → <code>/courses?id={id}</code>
Изменение схемы БД	Добавление новых колонок с NULL	<code>ALTER TABLE course ADD COLUMN rating INT DEFAULT NULL</code>
Изменение бизнес-логики	Новый сервис + деприкейшн старого	<code>@Deprecated</code> на старом методе

2.3. Анти-паттерны в AI-сгенерированном коде веб-приложений

Анти-паттерн	Проявление	Корректировка
Толстый контроллер	Бизнес-логика и валидация в контроллере	Вынос в Service, использование DTO
Игнорирование транзакций	Отсутствие <code>@Transactional</code> при нескольких операциях с БД	Добавление <code>@Transactional</code> на методы Service
N+1 запросы	Загрузка связанных сущностей в цикле	Использование <code>JOIN FETCH</code> или <code>@EntityGraph</code>
Отсутствие пагинации	Возврат <code>List<Entity></code> без ограничений	Использование <code>Pageable</code> и <code>Page<T></code>
Хранение секретов	API-ключи и пароли в коде	Вынос в <code>application.yml</code> и переменные окружения

3. ПОСТАНОВКА ЗАДАЧИ (Пример кейса)

3.1. Легенда

Вы — **Java-разработчик** в команде, создающей веб-приложение для онлайн-курсов дополнительного образования «EduPlus». Приложение включает функционал управления курсами, студентами, преподавателями и записями на курсы.

В рамках вайб-программирования с использованием корпоративного AI-ассистента вы начали реализацию функционала **просмотра и записи на курсы**. Однако **сегодня утром пришло обновление требования от продуктового владельца**. Вам нужно оценить ситуацию, скорректировать код, переформулировав промпты для AI, и не допустить хаоса.

3.2. Исходное ТЗ (версия 1.0, вчерашнее)

«Реализовать функционал для работы с курсами:

1. **Просмотр списка курсов:** GET `/api/courses` — возвращает все курсы с полями: `id`, `название`, `описание`, `цена`, `количество мест`.
2. **Запись студента на курс:** POST `/api/courses/{courseId}/enroll/{studentId}` — записывает студента на курс, если есть свободные места.
3. **Просмотр студентов курса:** GET `/api/courses/{courseId}/students` — возвращает список студентов, записанных на курс.

При записи на курс необходимо проверять, что студент ещё не записан на этот курс. Если мест нет — возвращать ошибку 409 Conflict.»

3.3. Обновлённое ТЗ (версия 2.0, пришло сегодня утром)

«Доработать функционал работы с курсами:

1. Просмотр списка курсов:

- Добавить **пагинацию** и **сортировку** по цене и дате создания.
- Добавить **фильтрацию** по категории, цене (`min/max`), наличию свободных мест.
- Добавить поле **рейтинг курса** (средняя оценка от студентов, доступно только для курсов, на которые есть хотя бы одна запись).

2. Запись студента на курс:

- Сохранить **обратную совместимость** со старым API (`v1`), но реализовать **новую версию `v2`**, где параметры передаются как **query-параметры**: POST `/api/v2/enroll?courseId={courseId}&studentId={studentId}`.
- Добавить **проверку прав доступа**: студент может записаться только на курсы, соответствующие его уровню (`Beginner / Intermediate / Advanced`).
- Добавить **бизнес-правило**: студент не может записаться более чем на 5 активных курсов одновременно.
- Добавить **rate limiting**: не более 3 запросов на запись в минуту на одного студента.

3. Просмотр студентов курса:

- Добавить **пагинацию**.
- Добавить возможность получения **только активных студентов** (тех, кто ещё не завершил курс) через **query-параметр `?activeOnly=true`**.

4. Добавить новый endpoint:

- **POST** `/api/v2/courses` — создание нового курса преподавателем (только для пользователей с ролью TEACHER или ADMIN).
- **GET** `/api/v2/courses/{id}/rating` — получение рейтинга курса.

Важно: все изменения API должны быть документированы в OpenAPI. Изменения схемы БД должны выполняться через миграции (Flyway) без потери данных. Обратная совместимость API v1 должна быть сохранена до следующего релиза.»

3.4. Фрагмент `project-rules.md` (обязателен к соблюдению)

1. Слоистая архитектура:

- `Controller` — только маршрутизация и валидация входных данных.
- `Service` — бизнес-логика, транзакционная (`@Transactional`).
- `Repository` — работа с БД через Spring Data JPA.
- `DTO` — для передачи данных между слоями (не использовать Entity напрямую в контроллерах).

2. Работа с БД:

- Запрещено использовать `@ManyToMany` и `@OneToMany` с `FetchType.EAGER` — только LAZY с явными JOIN в запросах.
- Все запросы к БД должны использовать пагинацию через `Pageable`.
- Для сложных запросов использовать `@Query` или `QueryDSL`.

3. Обработка ошибок:

- Использовать глобальный `@ControllerAdvice` для обработки исключений.
- Для бизнес-ошибок использовать кастомные исключения (например, `CourseFullException`, `StudentAlreadyEnrolledException`).

4. Безопасность:

- Использовать Spring Security, роли: STUDENT, TEACHER, ADMIN.
- API-ключи и секреты не хранятся в коде.
- Все endpoint'ы должны быть защищены (кроме публичных, если явно не указано).

5. Миграции:

- Все изменения схемы БД через Flyway (`V1__initial.sql`, `V2__add_rating.sql`, ...).
- Запрещено удалять колонки без явного согласования.

3.5. Что уже написал студент с помощью AI (по старому ТЗ v1.0)

№	Артефакт	Статус
1	<code>CourseController</code> с методами: <code>GET /api/courses</code> , <code>POST /api/courses/{courseId}/enroll/{studentId}</code> , <code>GET /api/courses/{courseId}/students</code>	☑ Готово
2	<code>CourseService</code> с методами: <code>getAllCourses()</code> , <code>enrollStudent()</code> , <code>getStudentsByCourse()</code>	☑ Готово
3	<code>CourseRepository</code> с <code>findAll()</code>	☑ Готово

№	Артефакт	Статус
4	StudentRepository с findById()	✓ Готово
5	EnrollmentRepository с методом existsByCourseAndStudent()	✓ Готово
6	Entity: Course, Student, Enrollment (без дополнительных полей)	✓ Готово
7	Миграция Flyway: V1__initial_schema.sql (таблицы courses, students, enrollments)	✓ Готово
8	3 unit-теста для CourseService	✓ Готово
9	Работа над правами доступа	✗ Не начата
10	OpenAPI-спецификация	✗ Не начата

3.6. Промпты, которые использовал студент для v1.0

Промпт №1:

«Напиши Spring Boot REST контроллер для курсов с тремя endpoint'ами: получение всех курсов, запись студента на курс, получение студентов курса. Используй Spring Data JPA. Создай три сущности: Course, Student, Enrollment. Для записи проверяй наличие мест и существование студента.»

Промпт №2:

«Создай миграцию Flyway для таблиц courses, students, enrollments. Добавь базовую связь многие-ко-многим через промежуточную таблицу. Напиши unit-тесты для сервиса.»

4. ЗАДАНИЯ ЛАБОРАТОРНОЙ РАБОТЫ

ЗАДАНИЕ 1. Анализ разрыва требований

Сформулируйте изменения в ТЗ.

Составьте таблицу сравнения старого (v1.0) и нового (v2.0) ТЗ. Для **каждого** изменения укажите:

- Что именно изменилось;

- Критичность изменения (высокая / средняя / низкая);
- Что это значит для уже сгенерированного кода.

Методическое указание: сравнивайте ТЗ построчно. Обратите внимание на изменения, которые не явно упомянуты, но следуют из контекста (например, добавление поля rating в Course требует миграции БД).

Пример начала таблицы:

№	Изменение	Критичность	Что это значит для уже сгенерированного кода
1	Добавлена пагинация и сортировка для GET /api/courses	Высокая	Метод getAllCourses() должен быть изменён на getCourses(Pageable pageable). Нужно добавить параметры в контроллер.
2	...	...	...

ЗАДАНИЕ 2. Формулировка промптов для рефакторинга

Напишите **новые промпты** для AI-ассистента, которые позволят модифицировать существующий код в соответствии с новым ТЗ. **Важно:** промпты должны быть направлены на **изменение существующего кода**, а не на его написание с нуля.

Требования к промптам:

1. **Контекст:** что уже есть, что работает.
2. **Явное указание на изменение:** «Требование изменилось, нужно сделать следующее...».
3. **Конкретные инструкции по изменению:** какие классы, методы, Entity нужно модифицировать.
4. **Что нельзя трогать:** ограничения (например, не менять сигнатуру v1 методов).
5. **Новые Acceptance Criteria.**

Минимальное количество промптов: минимум 3-4 (например, для изменения контроллера/сервиса, для миграции БД, для добавления безопасности, для тестов).

ЗАДАНИЕ 3. Таблица «Что оставляем / переделываем / удаляем»

Для **каждого** артефакта из п. 3.5 определите дальнейшие действия:

Артефакт	Оставляем	Переделываем	Удаляем	Обоснование (ссылка на правило / ТЗ)
...	...	...	...	...

Методическое указание: обоснование должно ссылаться на конкретные пункты `project-rules.md` или нового ТЗ.

ЗАДАНИЕ 4. Обновлённые тестовые сценарии

Составьте список тестовых сценариев с учётом **нового** ТЗ. Обязательно включите все категории:

Категория	Минимум сценариев
Позитивные сценарии (успешное выполнение)	3
Негативные сценарии (ошибки валидации, бизнес-правила)	3
Сценарии пагинации и фильтрации	2
Сценарии проверки прав доступа	3
Сценарии rate limiting	2

Для каждого сценария укажите: №, описание, тип, ожидаемый результат (HTTP-код и поведение).

ЗАДАНИЕ 5. Отчёт об использовании AI-инструментов

Заполните шаблон отчёта (раздел 7), зафиксировав:

- какие AI-инструменты использовались;
- какие запросы были сформулированы;
- какие ответы AI были приняты, изменены или отклонены;
- какие ошибки AI были обнаружены;
- как вы проверили итоговый результат вручную.

5. МЕТОДИЧЕСКИЕ УКАЗАНИЯ К ВЫПОЛНЕНИЮ

5.1. Рекомендуемый порядок работы

1. Прочитайте оба ТЗ и `project-rules.md`.
2. Выполните задание 1: анализ разрыва требований.
3. Выполните задание 3: решите, что оставляете, переделываете или удаляете.
4. Выполните задание 2: напишите промпты для рефакторинга.
5. Выполните задание 4: составьте тестовые сценарии.
6. Выполните задание 5: заполните отчёт.

5.2. Структура промпта для рефакторинга (шаблон для Задания 2)

****КОНТЕКСТ:****

У нас есть работающее веб-приложение для курсов дополнительного образования.

Реализованы: CourseController (v1), CourseService, CourseRepository, StudentRepository, EnrollmentRepository, Entity (Course, Student, Enrollment), Flyway миграция V1.

****ЧТО УЖЕ РАБОТАЕТ:****

- GET /api/courses — возвращает все курсы
- POST /api/courses/{courseId}/enroll/{studentId} — запись на курс
- GET /api/courses/{courseId}/students — список студентов курса

****ЧТО ИЗМЕНИЛОСЬ В ТРЕБОВАНИЯХ:****

- Было: возврат всех курсов без пагинации.
- Стало: добавить пагинацию, сортировку, фильтрацию.
- Было: запись через path-параметры.
- Стало: новая версия v2 с query-параметрами (сохранить обратную совместимость).
- ... (перечислить все изменения)

****ЗАДАЧА:****

Переделать существующий код, чтобы он соответствовал новым требованиям. Не создавай новый код с нуля, модифицируй существующий.

****КОНКРЕТНЫЕ ИЗМЕНЕНИЯ:****

1. В CourseController:

- Изменить GET /api/courses на GET /api/v1/courses (для обратной совместимости)
- Добавить GET /api/v2/courses с параметрами: page, size, sort, category, minPrice, maxPrice, hasAvailableSpots
- Добавить POST /api/v2/courses для создания курса (только TEACHER/ADMIN)
- ...

2. В CourseService:

- Изменить метод getAllCourses() на findAll(Pageable pageable, ...)
- Добавить метод createCourse(CourseCreateRequest request)
- ...

3. В Entity:

- Добавить поле rating в Course (Double)
- Добавить поле level в Student (enum: BEGINNER, INTERMEDIATE, ADVANCED)
- Добавить поле status в Enrollment (enum: ACTIVE, COMPLETED)
- ...

4. Создать миграцию V2__add_new_fields.sql

****ЧТО НЕЛЬЗЯ ТРОГАТЬ:****

- Не менять сигнатуру v1 методов (кроме добавления @Deprecated).
- Не удалять существующие колонки из БД.
- Не менять существующие связи между Entity без согласования.

****ACCEPTANCE CRITERIA:****

- [] GET /api/v2/courses возвращает Page<Course> с пагинацией.
- [] POST /api/v2/enroll?courseId=X&studentId=Y работает как v1, но с дополнительными проверками.
- [] Студент с уровнем BEGINNER не может записаться на курс уровня ADVANCED.
- [] Студент не может записаться более чем на 5 активных курсов.
- [] Rate limiting: 3 запроса в минуту.
- [] Все изменения прошли миграцию БД без потери данных.
- [] OpenAPI обновлён.

5.3. Разрешённые и запрещённые способы использования AI

Разрешено:

- ☒ Использовать AI для генерации **черновика** анализа разрыва требований.
- ☒ Использовать AI для подготовки **первого варианта** промптов для рефакторинга.
- ☒ Использовать AI для генерации **черновика** тестовых сценариев.
- ☒ Использовать AI как **помощника** при формулировании плана рефакторинга.

Запрещено:

- ☒ Просить AI «автоматически переделать весь код» без понимания причин.
- ☒ Сдавать AI-generated решения **без ручной проверки**.
- ☒ Передавать в AI реальные секретные данные.
- ☒ Копировать ответ AI без анализа и корректировки.

Принцип: финальное решение по рефакторингу принимает **студент**. Ответственность за результат остаётся у **студента**.

5.4. Чек-лист самопроверки перед сдачей

- ☐ Таблица разрыва содержит **все** изменения между v1.0 и v2.0.
- ☐ Каждое изменение имеет **критичность** и **обоснование**.
- ☐ Промпты **явно** указывают на изменение требования.
- ☐ Промпты содержат разделы: «контекст», «что меняем», «что не трогаем».
- ☐ Acceptance Criteria включают **все** пункты нового ТЗ.
- ☐ Таблица артефактов покрывает **все пункты** из п. 3.5.
- ☐ Обоснования ссылаются на `project-rules.md`.
- ☐ Тестовые сценарии покрывают **все категории** из п. 4 (Задание 4).

- ☐ Отчёт об использовании AI заполнен полностью.
- ☐ Нет признаков «слепого копирования» AI-ответов.

6. ШАБЛОН ОТЧЁТА

Титульная страница
(ФИО, группа, вариант, дата)

Таблица 1. Анализ разрыва требований (Задание 1)

№	Изменение	Критичность	Что это значит для уже сгенерированного кода
1			
2			

Промпты для AI-ассистента (Задание 2)

Промпт №1 (для изменения контроллера и сервиса):

[Вставьте текст]

Промпт №2 (для миграции БД и Entity):

[Вставьте текст]

Промпт №3 (для добавления безопасности и rate limiting):

[Вставьте текст]

Промпт №4 (для тестов):

[Вставьте текст]

Таблица 2. Что оставляем / переделываем / удаляем (Задание 3)

Артефакт	Оставляем	Переделываем	Удаляем	Обоснование (ссылка на правило / ТЗ)
CourseController				
CourseService				

Артефакт	Оставляем	Переделываем	Удаляем	Обоснование (ссылка на правило / ТЗ)
CourseRepository				
StudentRepository				
EnrollmentRepository				
Entity: Course				
Entity: Student				
Entity: Enrollment				
Миграция V1				
3 unit-теста				

Таблица 3. Обновлённые тестовые сценарии (Задание 4)

№	Сценарий	Тип	Ожидаемый результат
1			
2			

Отчёт об использовании AI-инструментов (Задание 5)

7.1. Используемые AI-инструменты

Название инструмента	Версия	Цель использования

7.2. Соответствие разрешённым способам использования

Отметьте галочками задействованные сценарии:

- ☐ Генерация черновика анализа разрыва требований. Краткое описание:

- ☐ Подготовка первого варианта обновлённого промпта. Краткое описание:

- ☐ Генерация тестовых сценариев. Краткое описание:

- ☐ Помощь при формулировании плана рефакторинга. Краткое описание:

7.3. Сформулированные запросы (промпты)

Запрос №1:

[Вставьте текст запроса]

Запрос №2:

[Вставьте текст запроса]

Запрос №3:

[Вставьте текст запроса]

7.4. Анализ ответов AI

№	Суть ответа AI	Решение студента (Принят / Изменён / Отклонён)	Обоснование решения
1			
2			

7.5. Обнаруженные ошибки и ограничения AI

Описание ошибки / некорректного поведения AI	Как была обнаружена	Способ исправления

7.6. Описание ручной проверки

Что проверено	Результат
Корректность логики (прогноз ключевые сценарии)	
Соответствие требованиям (сверил каждый пункт ТЗ)	
Отсутствие секретных данных в коде	

Что проверено	Результат
Работоспособность (запуск в тестовой среде)	
Проверка миграций БД (Flyway)	
Проверка обратной совместимости API v1	

7.7. Соблюдение ограничений

Ограничение	Соблюдено (Да/Нет)	Пояснение
Не просил AI автоматически переделать весь код		
Не сдал AI-generated решение без ручной проверки		
Не передавал в AI реальные секретные данные		
Финальное решение принял студент		

8. КРИТЕРИИ ОЦЕНКИ

Критерий	Баллы	Что оценивается
Анализ разрыва требований	0–20	Таблица полная, изменения классифицированы по критичности, обоснования корректны. Выявлены изменения в API, БД, бизнес-логике.
Формулировка промптов для рефакторинга	0–30	Промпты явно указывают на изменение, определяют что оставляем/переделываем, формулируют новые acceptance criteria, задают ограничения. Учтены слоистая архитектура и миграции БД.

Критерий	Баллы	Что оценивается
Таблица «Что оставляем / переделываем / удаляем»	0–25	Решения обоснованы, учтена обратная совместимость, распознаны проблемы старого кода (отсутствие пагинации, игнорирование прав доступа).
Обновлённые тестовые сценарии	0–15	Включены позитивные, негативные, сценарии пагинации/фильтрации, прав доступа, rate limiting.
Соблюдение правил работы с ИИ	0–10	Приложен лог промптов, показано, что студент верифицировал ответы.
ИТОГО	100	Зачёт при наборе > 70 баллов

9. КОНТРОЛЬНЫЕ ВОПРОСЫ

1. Почему при изменении требований нельзя просто «написать новый промпт для всего кода с нуля»? Какие риски это несёт для веб-приложения с БД?
2. В чём разница между использованием `@Transactional` на уровне контроллера и на уровне сервиса? Где правильно и почему?
3. Какие стратегии обеспечения обратной совместимости API вы знаете? Какую выбрали в своём решении и почему?
4. Почему важно разделять DTO и Entity? Какую проблему решает этот подход при изменении требований?
5. Как вы проверили, что AI-сгенерированный код не нарушает архитектурные правила (например, не содержит бизнес-логику в контроллере)?
6. Почему миграции БД (Flyway) критичны при изменении требований? Как избежать потери данных при добавлении новых полей?
7. Какие риски возникают при добавлении поля `rating` в сущность `Course`? Как обеспечить, чтобы старые курсы не «сломались»?
8. Приведите пример «лишней инициативы» AI-ассистента в контексте веб-приложения и как вы её предотвратили.